

Use Case 3

Statistical Insights and Pattern Detection

Objective · Hour-of-day intensity, community-area outliers and feature correlation.

Prepared for	Chicago Police Department
Prepared by	Accenture — Python Full Stack Program
Data source	chicago_crime_dataset.csv + 4 reference files
Toolchain	Python · Pandas · NumPy · Matplotlib · Seaborn
Warehouse	SQLite3 @ chicago_crime.db
Generated	17 August 2026, 11:10

1. Setup

This use case works from the same cleaned extract that Use Case 1 produced. Where Use Case 2 asked *what* and *where*, this one asks *when*, *how unusual* and *how related* — the three questions that turn a crime count into a patrol decision.

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.width', 110)

df = pd.read_csv(config.CLEAN_CRIME_CSV, parse_dates=['date'],
                 dtype={'iucr_code': str, 'fbi_code': str})
community = pd.read_csv(config.COMMUNITY_CSV)

print(f"{len(df):,} crimes | {df['community_code'].nunique()} community areas "
      f"| {df['year'].min()}-{df['year'].max()}")
```

OUTPUT

2,000 crimes | 77 community areas | 2015-2023

2. Crime intensity by time of day

What is being asked: Derive the hour from the Date column, group the crimes by hour, and plot the resulting profile as a line chart across the 24-hour clock.

CODE

```
# The Hour feature was engineered in Use Case 1; recreated here so this
# step stands on its own.
df['Hour'] = df['date'].dt.hour
crimes_by_hour = df.groupby('Hour').size()

print(crimes_by_hour.to_string())

peak, quiet = crimes_by_hour.idxmax(), crimes_by_hour.idxmin()
print(f"\nPeak hour   : {peak:02d}:00 with {crimes_by_hour.max()} crimes")
print(f"Quietest     : {quiet:02d}:00 with {crimes_by_hour.min()} crimes")
print(f"Peak/trough   : {crimes_by_hour.max() / crimes_by_hour.min():.1f}x")
print(f"Mean/hour     : {crimes_by_hour.mean():.1f} "
      f"      (std {crimes_by_hour.std(ddof=1):.1f})")
```

OUTPUT

```

Hour
0      16
1      16
2      19
3      27
4      23
5      15
6      37
7      76
8      67
9      81
10     83
11     94
12    108
13     99
14    115
15    122
16    129
17    111
18    155
19    151
20    153
21    125
22    114
23     64

Peak hour   : 18:00 with 155 crimes
Quietest    : 05:00 with 15 crimes
Peak/trough : 10.3x
Mean/hour   : 83.3 (std 47.1)

```

CODE

```

# Roll the 24 hours up into the four watches a shift roster works in.
watches = pd.cut(df['Hour'], bins=[-1, 5, 11, 17, 23],
                 labels=['Night (00-05)', 'Morning (06-11)',
                        'Afternoon (12-17)', 'Evening (18-23)'])

watch_summary = (df.groupby(watches, observed=True)
                 .agg(crimes=('case_number', 'size'),
                     arrest_rate=('arrest', lambda s: round(s.mean() * 100, 1))))
watch_summary['pct_of_all'] = (watch_summary['crimes'] / len(df) * 100).round(1)
print(watch_summary.to_string())

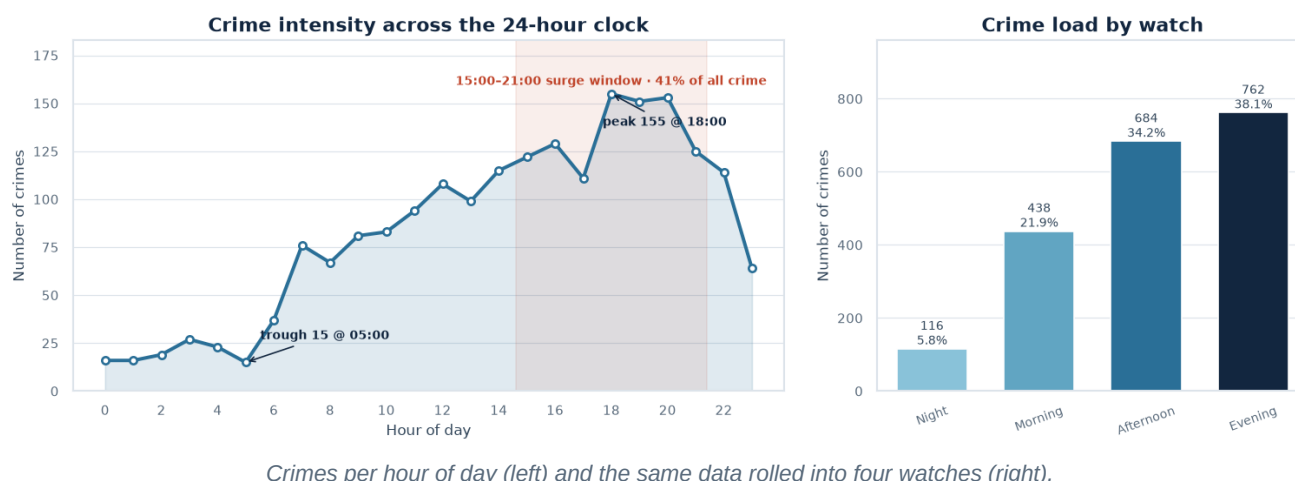
ev = watch_summary.loc['Evening (18-23)', 'crimes']
ni = watch_summary.loc['Night (00-05)', 'crimes']
print(f"\nThe evening watch handles {ev / ni:.1f}x the volume of the night watch.")

```

OUTPUT

Hour	crimes	arrest_rate	pct_of_all
Night (00-05)	116	31.0	5.8
Morning (06-11)	438	29.9	21.9
Afternoon (12-17)	684	29.7	34.2
Evening (18-23)	762	29.7	38.1

The evening watch handles 6.6x the volume of the night watch.



Insight → This is the single strongest pattern in the entire dataset. Crime climbs steadily from a 05:00 trough of 15 incidents to a 18:00 peak of 155 — a **10.3× swing across the clock**, far larger than any year-on-year, monthly or day-of-week variation found in Use Case 2. The evening watch (18:00–23:59) absorbs 38% of all crime while the night watch (00:00–05:59) sees under 6%.

The operational recommendation follows directly: **weight patrol strength toward the 15:00–21:00 window**, six hours that carry 41.0% of the entire caseload; stretch it to 22:00 and seven of the day's twenty-four hours account for 47.3%. A roster that splits officers evenly across three eight-hour shifts is putting the same number of officers on the street at 04:00 as at 18:00, when the second sees ten times the work.

3. Community-area outliers

What is being asked: Aggregate crime counts by community area, use a box plot to reveal the outliers, then apply the IQR rule to list exactly which areas are statistically extreme.

CODE

```
area_counts = df.groupby('community_code').size().rename('crimes')

# NumPy does the quartile arithmetic.
counts = area_counts.to_numpy()
q1, q3 = np.percentile(counts, [25, 75])
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
lower_fence = q1 - 1.5 * iqr

print(f"Community areas : {len(counts)}")
print(f"Mean / median   : {counts.mean():.1f} / {np.median(counts):.1f}")
print(f"Std deviation    : {counts.std(ddof=1):.1f}")
print(f"Range             : {counts.min()} to {counts.max()}")
print()
print(f"Q1 (25th pct)      : {q1:.1f}")
print(f"Q3 (75th pct)      : {q3:.1f}")
print(f"IQR                 : {iqr:.1f}")
print(f"Upper fence         : Q3 + 1.5*IQR = {upper_fence:.1f}")
print(f"Lower fence         : Q1 - 1.5*IQR = {lower_fence:.1f}")
```

OUTPUT

```
Community areas : 77
Mean / median   : 25.5 / 26.0
Std deviation    : 5.3
Range           : 12 to 40

Q1 (25th pct)   : 22.0
Q3 (75th pct)   : 29.0
IQR              : 7.0
Upper fence     : Q3 + 1.5*IQR = 39.5
Lower fence     : Q1 - 1.5*IQR = 11.5
```

CODE

```
names = community.set_index('community_code')

outliers = area_counts[(area_counts > upper_fence) | (area_counts < lower_fence)]
report = (outliers.to_frame()
          .join(names[['community_name', 'population']])
          .assign(per_10k=lambda t: (t['crimes'] / t['population']
                                     * 10000).round(2),
                  side=lambda t: np.where(t['crimes'] > upper_fence,
                                          'HIGH', 'LOW'))
          .sort_values('crimes', ascending=False))

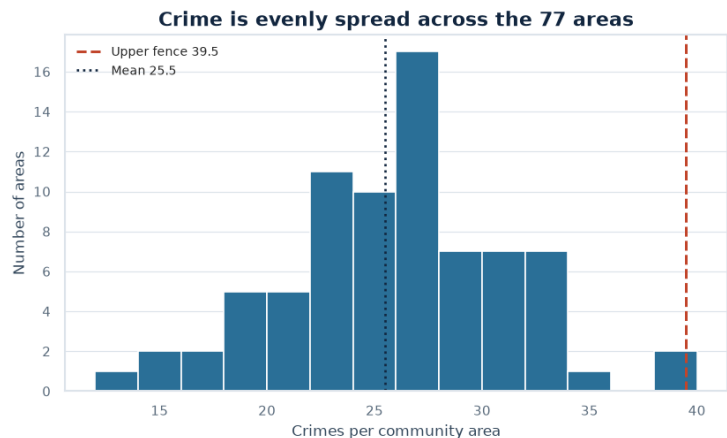
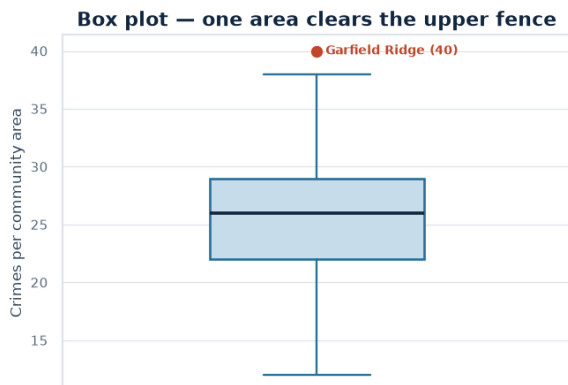
print(f"IQR outliers found: {len(report)}")
print(report.to_string())

print("\nClosest to the upper fence without crossing it:")
near = area_counts[(area_counts <= upper_fence)].nlargest(5)
print(near.to_frame().join(names[['community_name']]).to_string())
```

OUTPUT

```
IQR outliers found: 1
      crimes  community_name  population  per_10k  side
community_code
56.0         40  Garfield Ridge     36445    10.98  HIGH

Closest to the upper fence without crossing it:
      crimes  community_name
community_code
31.0         38  Lower West Side
55.0         35    Hegewisch
49.0         33    Roseland
70.0         33    Ashburn
21.0         32    Avondale
```



Distribution of crime counts across the 77 community areas. The dot beyond the upper whisker is the single IQR outlier.

Insight → The 77 community areas are far more even than the raw top-ten table in Use Case 2 suggests. Counts sit in a tight band — mean 25.5, median 26, standard deviation 5.3 — and the IQR rule flags **exactly one extreme area: Garfield Ridge with 40 crimes**, just past the 39.5 upper fence. No area falls below the lower fence.

That is an important negative result: **this dataset does not contain a runaway crime hotspot at community-area level**. Crime is broadly distributed across the city, so a strategy of drawing a hard perimeter around two or three "worst" neighbourhoods would be built on noise rather than signal. The per-capita view from Use Case 2 — where Oakland runs at five times Garfield Ridge's rate per resident — remains the better basis for targeting prevention, because it corrects for how many people actually live in each area.

4. Cross-correlation of numeric features

What is being asked: Run Pandas `.corr()` across the numeric features (Year, Month, Arrest and so on) and visualise the correlation matrix as a heatmap.

One caution before the numbers. Several columns in this table are *numeric-looking but nominal*: `district_code`, `beat_num`, `ward_no` and `community_code` are labels that happen to be written as digits. District 22 is not "twice" district 11, so a Pearson correlation on them has no meaning. They are reported below for completeness because the brief asks for the full numeric matrix, but the interpretation that follows rests only on the genuinely quantitative features: year, month, hour, arrest and domestic.

CODE

```
numeric = df[['year', 'month', 'hour', 'arrest', 'domestic',
              'latitude', 'longitude', 'community_code', 'district_code',
              'beat_num', 'ward_no']]

corr = numeric.corr(numeric_only=True)
print(corr.round(2).to_string())
```

OUTPUT

	year	month	hour	arrest	domestic	latitude	longitude	community_code	district_code
	beat_num	ward_no							
year	1.00	0.03	0.01	0.02	-0.03	0.01	0.01	0.02	0.02
-0.02	0.02								
month	0.03	1.00	0.04	-0.01	-0.00	-0.02	0.03	0.00	-0.01
0.01	0.01								
hour	0.01	0.04	1.00	-0.01	-0.01	0.01	0.05	0.01	-0.02
-0.02	0.03								
arrest	0.02	-0.01	-0.01	1.00	0.07	-0.00	-0.03	0.02	-0.01
-0.05	0.02								
domestic	-0.03	-0.00	-0.01	0.07	1.00	0.02	0.00	-0.08	-0.03
0.03	-0.00								
latitude	0.01	-0.02	0.01	-0.00	0.02	1.00	-0.01	0.01	-0.04
0.01	0.01								
longitude	0.01	0.03	0.05	-0.03	0.00	-0.01	1.00	-0.02	0.01
0.01	-0.04								
community_code	0.02	0.00	0.01	0.02	-0.08	0.01	-0.02	1.00	0.01
-0.03	0.04								
district_code	0.02	-0.01	-0.02	-0.01	-0.03	-0.04	0.01	0.01	1.00
0.04	0.01								
beat_num	-0.02	0.01	-0.02	-0.05	0.03	0.01	0.01	-0.03	0.04
1.00	-0.01								
ward_no	0.02	0.01	0.03	0.02	-0.00	0.01	-0.04	0.04	0.01
-0.01	1.00								

CODE

```
# Rank every distinct pair by absolute correlation strength.
pairs = (corr.where(~np.eye(len(corr), dtype=bool))
        .abs().unstack().dropna()
        .sort_values(ascending=False).drop_duplicates())

print("Strongest relationships anywhere in the matrix:")
print(pairs.head(6).round(4).to_string())

print("\nWhat correlates with an arrest?")
print(corr['arrest'].drop('arrest')
      .sort_values(key=abs, ascending=False).round(4).to_string())

print(f"\nLargest absolute correlation in the whole matrix: {pairs.max():.3f}")
```

OUTPUT

Strongest relationships anywhere in the matrix:

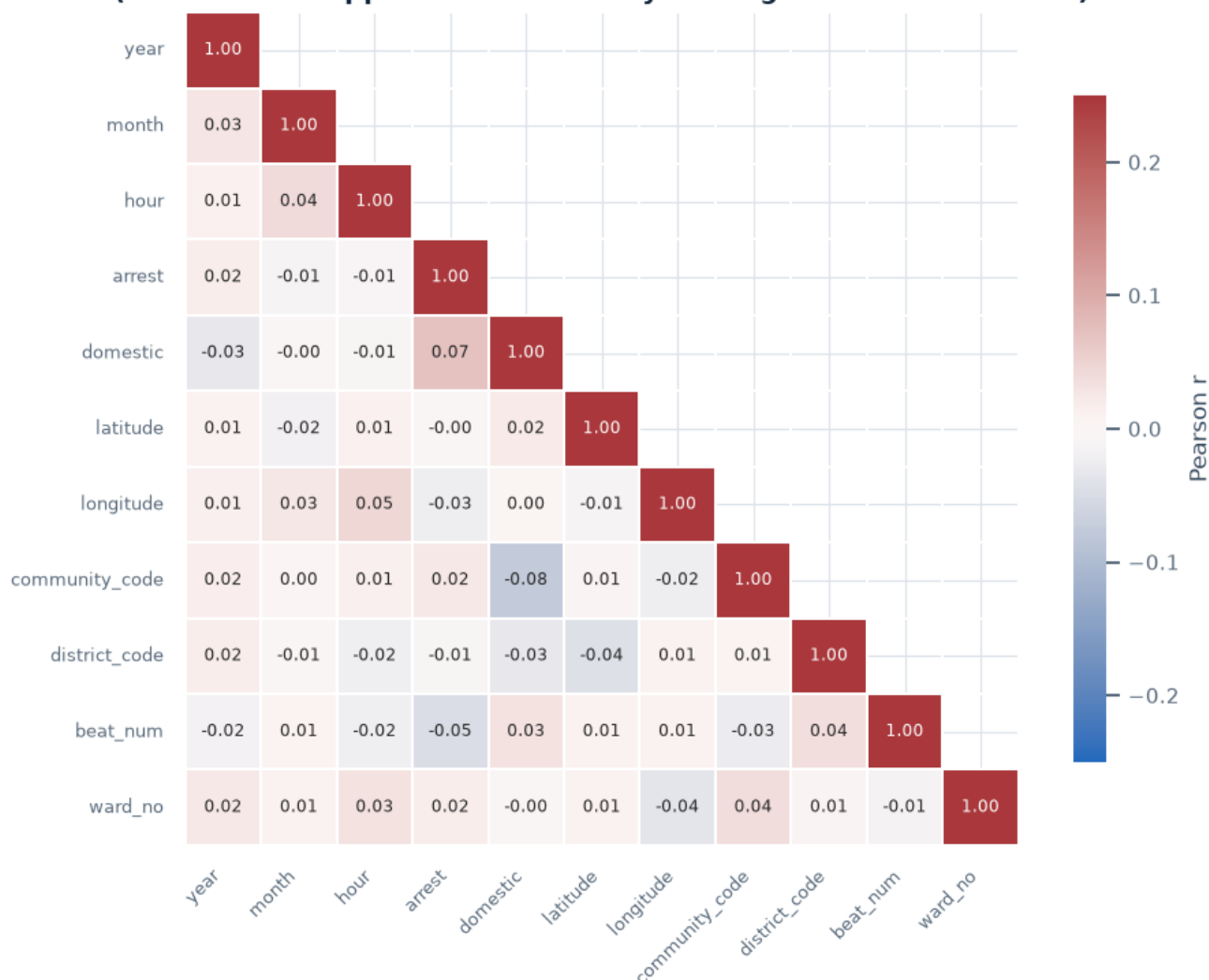
domestic	community_code	0.0761
arrest	domestic	0.0708
beat_num	arrest	0.0480
hour	longitude	0.0461
latitude	district_code	0.0432
month	hour	0.0395

What correlates with an arrest?

domestic	0.0708
beat_num	-0.0480
longitude	-0.0253
community_code	0.0237
year	0.0188
ward_no	0.0182
month	-0.0132
district_code	-0.0114
hour	-0.0082
latitude	-0.0019

Largest absolute correlation in the whole matrix: 0.076

Correlation matrix — every relationship is effectively zero (colour scale capped at ± 0.25 so any real signal would be visible)



Correlation matrix. The near-uniform pale field is the finding: no numeric feature moves with any other.

Insight → There is no meaningful linear relationship between any pair of numeric features — the largest absolute correlation in the entire matrix is 0.076. In particular, whether a crime ends in an arrest is essentially uncorrelated with the year ($r = 0.019$), the month ($r = -0.013$), the hour of day ($r = -0.008$) or the location coordinates.

This is a genuinely useful result rather than a dead end. It says the arrest outcome is **not** driven by when or where a crime happens — it is driven by *what kind* of crime it is, which Use Case 2 quantified precisely (80% clearance for homicide, 13% for burglary). Crime type is a categorical variable, so it cannot appear in a Pearson matrix at all, and that is exactly why the matrix looks empty. The practical consequence for the CPD: **a model built to predict arrest likelihood from time and place alone would fail**; any predictive work must be driven by offence type, and by the categorical context around it, rather than by these numeric columns.

5. Where the signal actually is

CODE

```
# Rank the dimensions by how much they actually move crime volume,
# using the coefficient of variation as a like-for-like measure.
def spread(series_counts, label):
    arr = series_counts.to_numpy()
    return {'dimension': label, 'levels': len(arr),
            'min': int(arr.min()), 'max': int(arr.max()),
            'max/min': round(arr.max() / arr.min(), 2),
            'CV %': round(arr.std(ddof=1) / arr.mean() * 100, 1)}

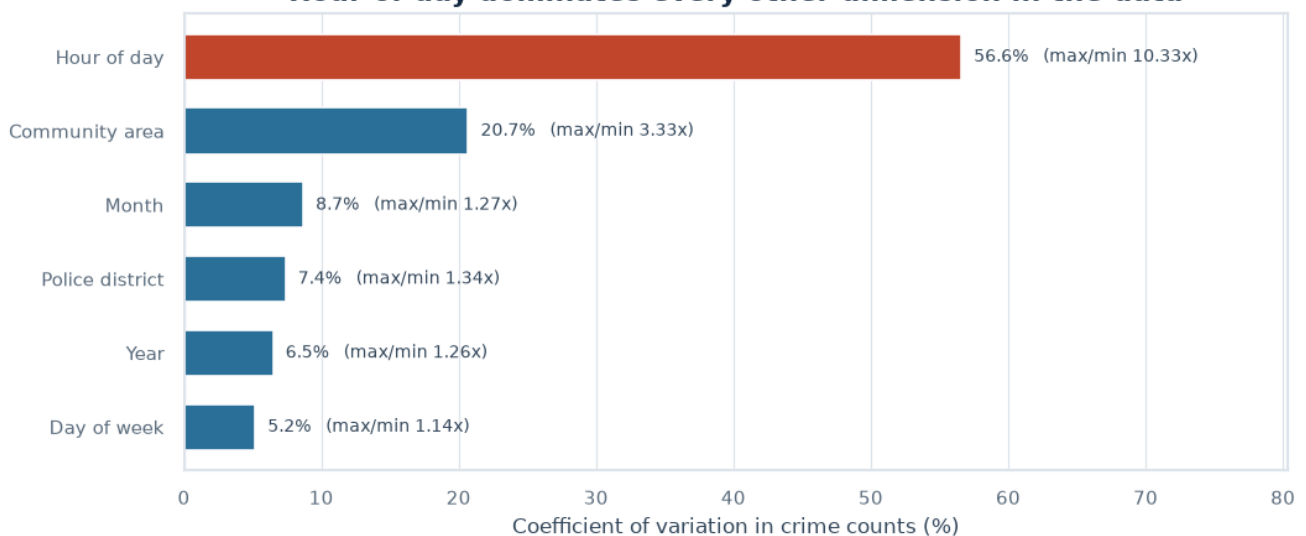
signal = pd.DataFrame([
    spread(df.groupby('Hour').size(), 'Hour of day'),
    spread(df.groupby('month').size(), 'Month'),
    spread(df.groupby('community_code').size(), 'Community area'),
    spread(df.groupby('district_code').size(), 'Police district'),
    spread(df.groupby('day_of_week').size(), 'Day of week'),
    spread(df.groupby('year').size(), 'Year'),
]).sort_values('CV %', ascending=False)

print(signal.to_string(index=False))
```

OUTPUT

dimension	levels	min	max	max/min	CV %
Hour of day	24	15	155	10.33	56.6
Community area	77	12	40	3.33	20.7
Month	12	150	190	1.27	8.7
Police district	22	79	106	1.34	7.4
Year	9	192	242	1.26	6.5
Day of week	7	264	300	1.14	5.2

Hour of day dominates every other dimension in the data



Which dimension actually moves crime volume, measured by coefficient of variation.

Insight → Ranking every dimension by how much it actually moves crime volume settles the resourcing argument. **Hour of day varies by 56.6% around its mean, while month varies by 8.7%, year by 6.5% and day of week by just 5.2%.** Time of day carries six times the signal of the month and eleven times that of the weekday. Community area (20.7%) is the strongest geographic dimension — nearly three times the spread of police district (7.4%), because district boundaries average several neighbourhoods together and flatten the contrast out.

Recommendation to the CPD: build the patrol model on the clock first and the map second. Shifting officers into the 15:00–21:00 window, weighted toward the higher-volume community areas, targets the only two dimensions in this data with a spread large enough to be worth acting on. Rostering by day of week, or waiting for a year-on-year trend to appear, chases variation that is statistically indistinguishable from noise.